Entrega — ALUControl y Banco de Registros

Qué hace la ALU y dónde se guardan los datos

Integrantes: Marcos López | Luis Angel Ramirez **Carnés: 231928 | 221165**

Machote de entrega · Partes del procesador MIPS: ALUControl y Banco de Registros

Dos ejercicios, cada uno con código, testbench comentado, capturas de consola y GTKWave, y casos de prueba.

Antes de empezar

Qué se entrega

Los nombres de los módulos, de los puertos y de los archivos los escoge cada grupo.

Módulo	Puertos	Qué hace
`ALUControl`	Entradas: ALUOp (2 bits) y funct (6 bits). Salida: alu_control (4 bits, reg).	Combina la señal de la Unidad de Control con el funct de las tipo R y decide la operación exacta de la ALU (lógica combinacional).
`Banco de Registros`	Entradas: reloj, reset (activo alto), RegWrite, dos direcciones de lectura y una de escritura (5 bits) y el dato de escritura (32 bits). Salidas: dos datos leídos (32 bits).	32 registros de 32 bits. Lectura asíncrona, escritura síncrona en el flanco ↑ y $0 siempre en 0.

Codificación que debe producir el ALUControl

Es la tabla del enunciado. Lo que no aparece aquí (funct o ALUOp no definidos) lo decide cada grupo y lo documenta.

Instrucción	ALUOp	funct	Operación	alu_control
MOVI	`00`	`X (no importa)`	MOVI	`0000`
ADD	`10`	`100000`	ADD	`0010`
SUB	`10`	`100100`	SUB	`0110`
MULT	`10`	`100001`	MULT	`1000`
DIV	`10`	`100010`	DIV	`1001`
MOV	`10`	`100011`	MOV	`0001`

Formato de salida sugerido

```
ALUOp=10, funct=100000 -> alu_control=0010
R1=10, R2=20, R3=30, R0=0
```

Detalles técnicos que evitan sorpresas

- **Sin latches.** En un always @(*) asigna un valor por defecto a alu_control antes del case. Así las combinaciones no cubiertas no dejan la salida en latch o en x.
- **El funct solo manda con ALUOp = 10.** Con ALUOp = 00 el funct no importa: pruébalo con un funct que sí esté en la tabla.
- **Reset del banco.** El enunciado dice «reinicio activo alto» pero no dice si es síncrono o asíncrono: escoge uno y documéntalo. Con reset síncrono hace falta un flanco ↑ con reset = 1; antes de eso los registros valen x.
- **El registro $0.** Dos protecciones: no escribir nunca en $0 y entregar 0 al leerlo, aunque el arreglo tenga x.

- **Leer registros desde el testbench.** El arreglo está dentro del módulo: se lee con el nombre de la instancia, por ejemplo instancia.registers[1] (el registers[1] del enunciado solo compila así). Imprime en el flanco ↓ para ver el valor ya actualizado.
- **Ondas.** Cada testbench lleva $dumpfile("nombre.vcd") y $dumpvars(0, nombre_del_tb).

```
iverilog -o salida.vvp modulo.v testbench.v  ->  vvp salida.vvp  ->  gtkwave nombre.vcd
```

Ejercicio 1 — ALUControl

1. Código Verilog de ALUControl

Traduce ALUOp y funct a la señal de 4 bits de la ALU. Comenta qué hace cada rama del case, por qué hay un valor por defecto y qué devuelve tu módulo en las combinaciones no definidas.

```
`timescale 1ns/1ns

// Decodifica la operacion solicitada.
// Los codigos funct siguen exactamente la tabla de la tarea.
module ALUControl (
    input  wire [1:0] ALUOp,
    input  wire [5:0] funct,
    output reg  [3:0] alu_control
);

    // Logica combinacional: responde a cambios en las entradas.
    always @(*) begin
        case (ALUOp)

            // MOVI: el campo funct no afecta esta seleccion.
            2'b00: alu_control = 4'b0000;

            // Instrucciones tipo R: decodificar funct.
            2'b10: begin
                case (funct)
                    6'b100000: alu_control = 4'b0010; // ADD
                    6'b100100: alu_control = 4'b0110; // SUB
                    6'b100001: alu_control = 4'b1000; // MULT
                    6'b100010: alu_control = 4'b1001; // DIV
                    6'b100011: alu_control = 4'b0001; // MOV

                    // Combinacion no definida en la tabla.
                    default: alu_control = 4'b1111;
                endcase
            end

            // ALUOp no definido en la tabla.
            default: alu_control = 4'b1111;

        endcase
    end

endmodule
```

El módulo ALUControl utiliza lógica combinacional mediante un bloque always @(*) y estructuras case. Cuando ALUOp es 00, genera el código 0000 correspondiente a MOVI, sin depender del campo funct. Cuando ALUOp es 10, utiliza funct para seleccionar ADD, SUB, MULT, DIV o MOV.

Para las combinaciones no definidas se eligió la salida 1111. Este valor identifica una combinación no contemplada en la tabla; no representa una operación adicional. Los casos default del case exterior e interior garantizan que siempre se asigne una salida, evitando la inferencia de latches.

2. Testbench de ALUControl

Aplica ADD, SUB, MULT, DIV, MOV y ADDI (y las combinaciones no definidas) y muestra entradas y salida. Verifica el valor esperado, no solo lo imprimas.

Tu testbench — ALUControl

```
`timescale 1ns/1ns

module tb_ALUControl;

    reg  [1:0] ALUOp;
    reg  [5:0] funct;
    wire [3:0] alu_control;

    integer errores;
    integer pruebas;

    ALUControl uut (
        .ALUOp(ALUOp),
        .funct(funct),
        .alu_control(alu_control)
    );

    // Aplica las entradas y compara la salida con el valor esperado.
    task probar;
        input [1:0] op;
        input [5:0] funcion;
        input [3:0] esperado;

        begin
            ALUOp = op;
            funct = funcion;
            #10;

            pruebas = pruebas + 1;

            $display(
                "ALUOp=%b, funct=%b -> alu_control=%b | esperado=%b",
```

```
                ALUOp, funct, alu_control, esperado
            );

            if (alu_control !== esperado) begin
                errores = errores + 1;
                $display("ERROR en prueba %0d", pruebas);
            end
        end
    endtask

    initial begin
        $dumpfile("ALUControl.vcd");
        $dumpvars(0, tb_ALUControl);

        errores = 0;
        pruebas = 0;

        $display("=== Pruebas de ALUControl ===");

        probar(2'b00, 6'b000000, 4'b0000); // MOVI
        probar(2'b10, 6'b100000, 4'b0010); // ADD
        probar(2'b10, 6'b100100, 4'b0110); // SUB
        probar(2'b10, 6'b100001, 4'b1000); // MULT
        probar(2'b10, 6'b100010, 4'b1001); // DIV
        probar(2'b10, 6'b100011, 4'b0001); // MOV

        // MOVI debe ignorar funct.
        probar(2'b00, 6'b100000, 4'b0000);
        // Caso rotulado ADDI en el formato: ALUOp = 00.
        // Segun la tabla implementada, produce 0000 e ignora funct.
        // Esta prueba verifica la decodificacion, no una suma inmediata.
        probar(2'b00, 6'b111111, 4'b0000);

        // Casos no definidos: deben producir nuestro codigo 1111.
        probar(2'b10, 6'b111111, 4'b1111);
        probar(2'b01, 6'b100000, 4'b1111);
        probar(2'b11, 6'b100000, 4'b1111);

        $display("=== Fin: pruebas=%0d, errores=%0d ===",
                 pruebas, errores);

        $finish;
    end

endmodule
```

El testbench aplica las operaciones de la tabla y compara cada salida con un valor esperado utilizando el operador !==, que también detecta diferencias causadas por valores desconocidos. Cada

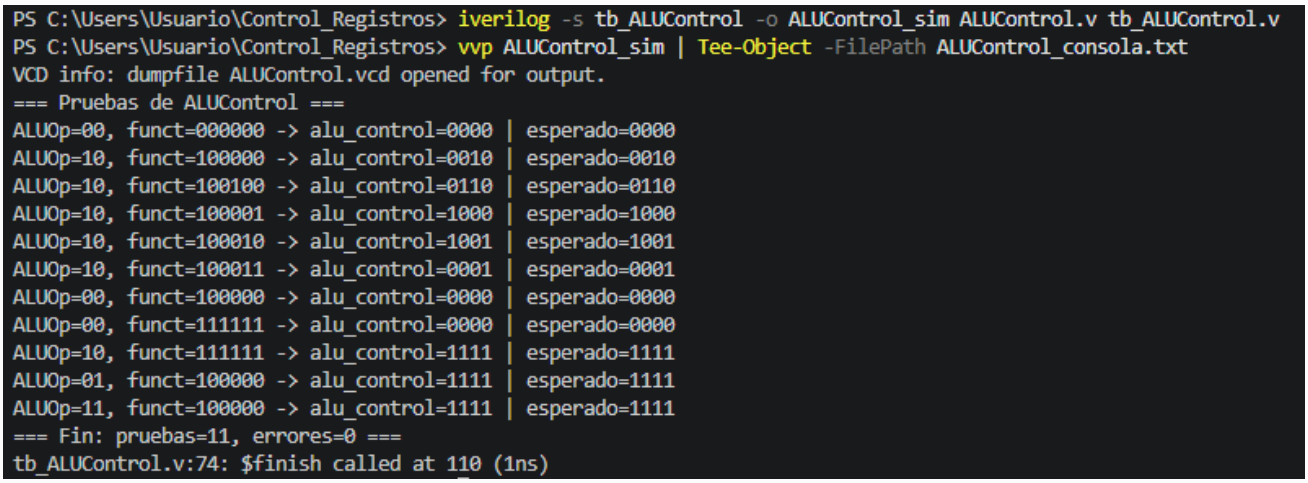
prueba muestra ALUOp, funct, alu_control y el resultado esperado mediante $display. Además, se comprueba que funct no modifica la salida cuando ALUOp es 00 y que las combinaciones no definidas producen 1111.

El formato de entrega identifica ADDI con ALUOp = 00, mientras que la tabla asigna esa misma entrada a MOVI con salida 0000. Por ello, el caso rotulado ADDI verifica la salida 0000 conforme a la tabla. Esto no demuestra la ejecución de una suma inmediata, ya que ALUControl solamente genera señales de control.

3. Captura de la consola

Debe verse el comando de compilación y ejecución, y una línea por prueba con ALUOp, funct y alu_control.

Captura — consola (ALUControl)

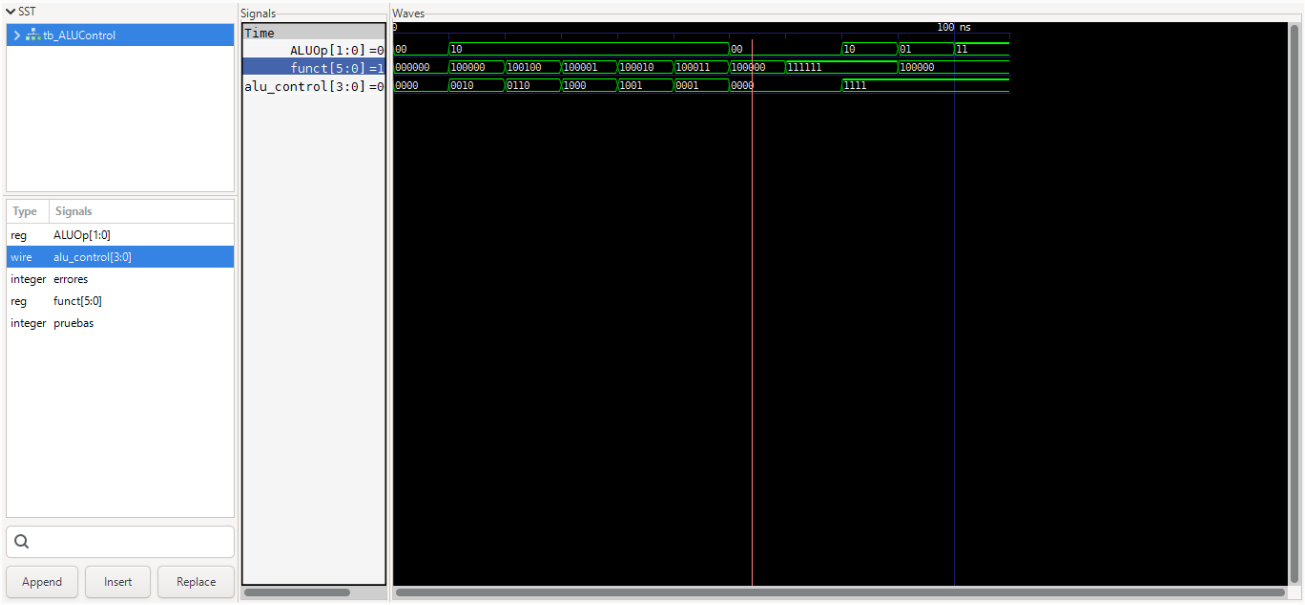


Compilación y ejecución del testbench de ALUControl. Se realizaron 11 pruebas y todas coincidieron con los valores esperados, sin errores.

4. Captura de GTKWave

ALUOp, funct y alu_control, con todas las pruebas a la vista (View → Zoom Fit). Muestra alu_control en binario.

Captura — GTKWave (ALUControl)



Señales de ALUControl durante la simulación. Cada prueba ocupa 10 ns. Se observan los códigos de las operaciones y la salida 1111 para combinaciones no definidas.

5. Detalle de los casos de prueba

Las entradas ya están anotadas donde el enunciado las fija; completa las que dicen «elige». Escribe el valor esperado a mano (según la tabla del enunciado) antes de simular y luego copia lo que dice la consola.

#	Caso de prueba	ALUOp	funct	alu_control esperado	alu_control obtenido (consola)	¿OK?
1	**ADD** *Tipo R con el funct de la suma*	10	100000	0010	0010	Sí
2	**SUB** *Tipo R con el funct de la resta*	10	100100	0110	0110	Sí
3	**MULT** *Tipo R con el funct de la multiplicación*	10	100001	1000	1000	Sí
4	**DIV** *Tipo R con el funct de la división*	10	100010	1001	1001	Sí
5	**MOV** *Tipo R con el funct de mover*	10	100011	0001	0001	Sí
6	**MOVI** *ALUOp = 00: el funct no importa, elige el que quieras*	00	000000	0000	0000	Sí
7	**ADDI** *Instrucción tipo I: llega con ALUOp = 00*	00	111111	0000	0000	Sí, para la decodificación indicada

#	Caso de prueba	ALUOp	funct	alu_control esperado	alu_control obtenido (consola)	¿OK?
8	**El funct se ignora con ALUOp = 00** *Pon el funct de la suma y comprueba que no cambia nada*	`00`	`100000`	0000	0000	Sí
9	**funct no definido** *ALUOp = 10 con un funct que no esté en la tabla. ¿Qué devuelve tu módulo?*	`10`	111111	1111	1111	Sí
10	**ALUOp no definido (01)** *¿Qué devuelve tu módulo?*	`01`	100000	1111	1111	Sí
11	**ALUOp no definido (11)** *¿Qué devuelve tu módulo?*	`11`	100000	1111	1111	Sí
12	**Caso adicional (opcional)**	No aplica	No aplica	No aplica	No ejecutado	No aplica

Ejercicio 2 — Banco de Registros

1. Código Verilog de el Banco de Registros

32 registros de 32 bits, lectura asíncrona, escritura síncrona en el flanco ↑ y $0 siempre en 0. Comenta cómo garantizas lo del $0, qué hace tu reset y por qué la lectura no depende del reloj.

Tu código — el Banco de Registros

```
`timescale 1ns/1ns

module RegisterFile (
    input wire clk,
    input wire reset,
    input wire RegWrite,
    input wire [4:0] read_reg1,
    input wire [4:0] read_reg2,
    input wire [4:0] write_reg,
    input wire [31:0] write_data,
    output wire [31:0] read_data1,
    output wire [31:0] read_data2
);

    // 32 registros, cada uno de 32 bits.
    reg [31:0] registers [0:31];
    integer i;

    // Lectura asincrona: no necesita un flanco del reloj.
    // Leer el registro cero siempre devuelve cero.
    assign read_data1 = (read_reg1 == 5'd0)
                      ? 32'd0 : registers[read_reg1];

    assign read_data2 = (read_reg2 == 5'd0)
                      ? 32'd0 : registers[read_reg2];

    // Reset y escritura sincronos.
    always @(posedge clk) begin
        if (reset) begin
            for (i = 0; i < 32; i = i + 1) begin
                registers[i] <= 32'd0;
            end
        end
        else begin
            registers[0] <= 32'd0;

            // Solo escribir si esta habilitado y no es R0.
            if (RegWrite && (write_reg != 5'd0)) begin
                registers[write_reg] <= write_data;
            end
        end
    end
```

```
endmodule
```

El banco contiene 32 registros de 32 bits declarados como registers[0:31]. Las lecturas se implementan mediante asignaciones continuas, por lo que las salidas responden a cambios de dirección sin esperar un flanco del reloj.

La escritura se realiza en el flanco positivo de clk cuando RegWrite es 1 y write_reg es distinto de cero. Se utilizan asignaciones no bloqueantes para actualizar los registros.

El reset es síncrono y activo alto: al llegar un flanco positivo con reset = 1, todos los registros se colocan en cero. El reset tiene prioridad sobre la escritura. Antes del primer flanco de reinicio, los registros distintos de cero pueden contener valores desconocidos en simulación.

El registro $0 se protege de dos formas: se impide escribir en la dirección cero y las salidas de lectura entregan cero cuando se selecciona dicha dirección, incluso antes del reinicio.

2. Testbench de el Banco de Registros

Aplica reset, escribe en varios registros, intenta escribir en $0 y lee por los dos puertos. Imprime los registros después de cada escritura y verifica los valores esperados.

Tu testbench — el Banco de Registros

```
`timescale 1ns/1ns

module tb_RegisterFile;

    reg clk;
    reg reset;
    reg RegWrite;
    reg [4:0] read_reg1;
    reg [4:0] read_reg2;
    reg [4:0] write_reg;
    reg [31:0] write_data;

    wire [31:0] read_data1;
    wire [31:0] read_data2;

    integer pruebas;
    integer errores;
    integer i;

    RegisterFile uut (
        .clk(clk),
        .reset(reset),
        .RegWrite(RegWrite),
        .read_reg1(read_reg1),
        .read_reg2(read_reg2),
```

```
        .write_reg(write_reg),
        .write_data(write_data),
        .read_data1(read_data1),
        .read_data2(read_data2)
    );

    // Periodo de reloj: 10 ns.
    always #5 clk = ~clk;

    // Comparar un valor con su resultado esperado.
    task verificar;
        input [31:0] obtenido;
        input [31:0] esperado;
        begin
            pruebas = pruebas + 1;

            if (obtenido !== esperado) begin
                errores = errores + 1;
                $display(
                    "ERROR prueba %0d: obtenido=%0d, esperado=%0d",
                    pruebas, obtenido, esperado
                );
            end
        end
    endtask

    // Escribir un dato y mostrar los registros actualizados.
    task escribir;
        input [4:0] direccion;
        input [31:0] dato;
        begin
            // Preparar entradas en el flanco negativo.
            @(negedge clk);
            RegWrite = 1;
            write_reg = direccion;
            write_data = dato;

            // La escritura ocurre en el flanco positivo.
            @(posedge clk);
            #1;

            $display(
                "Escritura R%0d <- %0d | R1=%0d R2=%0d R3=%0d R0=%0d",
                direccion, dato,
                uut.registers[1], uut.registers[2],
                uut.registers[3], uut.registers[0]
            );
```

```
            RegWrite = 0;
        end
    endtask

    // Leer y verificar ambos puertos sin esperar un flanco.
    task leer_y_verificar;
        input [4:0] direccion1;
        input [4:0] direccion2;
        input [31:0] esperado1;
        input [31:0] esperado2;
        begin
            read_reg1 = direccion1;
            read_reg2 = direccion2;
            #1;

            $display(
                "Lectura: R%0d=%0d | R%0d=%0d",
                direccion1, read_data1,
                direccion2, read_data2
            );

            verificar(read_data1, esperado1);
            verificar(read_data2, esperado2);
        end
    endtask

    initial begin
        $dumpfile("RegisterFile.vcd");
        $dumpvars(0, tb_RegisterFile);

        clk = 0;
        reset = 1;
        RegWrite = 0;
        read_reg1 = 0;
        read_reg2 = 0;
        write_reg = 0;
        write_data = 0;
        pruebas = 0;
        errores = 0;

        $display("=== Pruebas de RegisterFile ===");

        // 1. Reset sincronico y comprobacion de los 32 registros.
        @(posedge clk);
        #1;

        for (i = 0; i < 32; i = i + 1) begin
            verificar(uut.registers[i], 32'd0);
```

```
        end

        reset = 0;
        $display("Reset inicial: 32 registros comprobados.");

        // 2. Escrituras y lecturas normales.
        escribir(5'd1, 32'd10);
        leer_y_verificar(5'd1, 5'd0, 32'd10, 32'd0);

        escribir(5'd2, 32'd20);

        // Leer dos registros distintos simultaneamente.
        leer_y_verificar(5'd1, 5'd2, 32'd10, 32'd20);

        // Leer R2 por ambos puertos: agrega dos comprobaciones.
        leer_y_verificar(5'd2, 5'd2, 32'd20, 32'd20);

        // Escribir y comprobar R3.
        escribir(5'd3, 32'd30);
        leer_y_verificar(5'd3, 5'd0, 32'd30, 32'd0);

        // 3. Intentar escribir en R0: debe seguir en cero.
        escribir(5'd0, 32'd999);
        leer_y_verificar(5'd0, 5'd1, 32'd0, 32'd10);
        verificar(uut.registers[0], 32'd0);

        // 4. RegWrite = 0 debe impedir la escritura.
        @(negedge clk);
        RegWrite = 0;
        write_reg = 5'd2;
        write_data = 32'd777;

        @(posedge clk);
        #1;
        leer_y_verificar(5'd2, 5'd3, 32'd20, 32'd30);

        // 5. Comprobar el ultimo registro y un dato de 32 bits.
        escribir(5'd31, 32'hDEADBEEF);
        leer_y_verificar(5'd31, 5'd0, 32'hDEADBEEF, 32'd0);

        // 6. Comprobar R1 antes y despues del flanco de escritura.
        @(negedge clk);
        RegWrite = 1;
        write_reg = 5'd1;
        write_data = 32'd99;
        read_reg1 = 5'd1;

        #1;
```

```
        verificar(read_data1, 32'd10);
        $display(
            "Antes del flanco: R1=%0d, dato pendiente=%0d",
            read_data1, write_data
        );

        @(posedge clk);
        #1;
        verificar(read_data1, 32'd99);
        RegWrite = 0;
        $display("Escritura sincronica: R1=%0d", read_data1);

        // 7. Cambiar direcciones de lectura sin esperar otro flanco.
        leer_y_verificar(5'd2, 5'd3, 32'd20, 32'd30);
        leer_y_verificar(5'd1, 5'd31, 32'd99, 32'hDEADBEEF);

        // 8. Reset con datos: tiene prioridad sobre RegWrite.
        @(negedge clk);
        reset = 1;
        RegWrite = 1;
        write_reg = 5'd3;
        write_data = 32'd555;

        @(posedge clk);
        #1;

        for (i = 0; i < 32; i = i + 1) begin
            verificar(uut.registers[i], 32'd0);
        end

        reset = 0;
        RegWrite = 0;
        leer_y_verificar(5'd3, 5'd0, 32'd0, 32'd0);

        $display(
            "=== Fin: pruebas=%0d, errores=%0d ===",
            pruebas, errores
        );

        $finish;
    end

endmodule
```

El testbench genera un reloj con período de 10 ns y aplica un reset inicial. Después escribe y lee distintos registros, comprueba ambos puertos de lectura, intenta modificar $0 y verifica que RegWrite = 0 impide escribir.

También comprueba la lectura del mismo registro por ambos puertos, la escritura y lectura de $31, la sobrescritura de R1, el comportamiento antes y después del flanco de escritura y la prioridad del reset sobre RegWrite.

Las entradas de escritura se preparan en el flanco negativo. Los resultados se comprueban 1 ns después del flanco positivo para permitir que se completen las asignaciones no bloqueantes. Las lecturas asíncronas se verifican cambiando las direcciones y esperando 1 ns, sin esperar un nuevo flanco de reloj.

3. Captura de la consola

Debe verse el comando de compilación y ejecución, y los registros impresos después de cada ciclo de escritura.

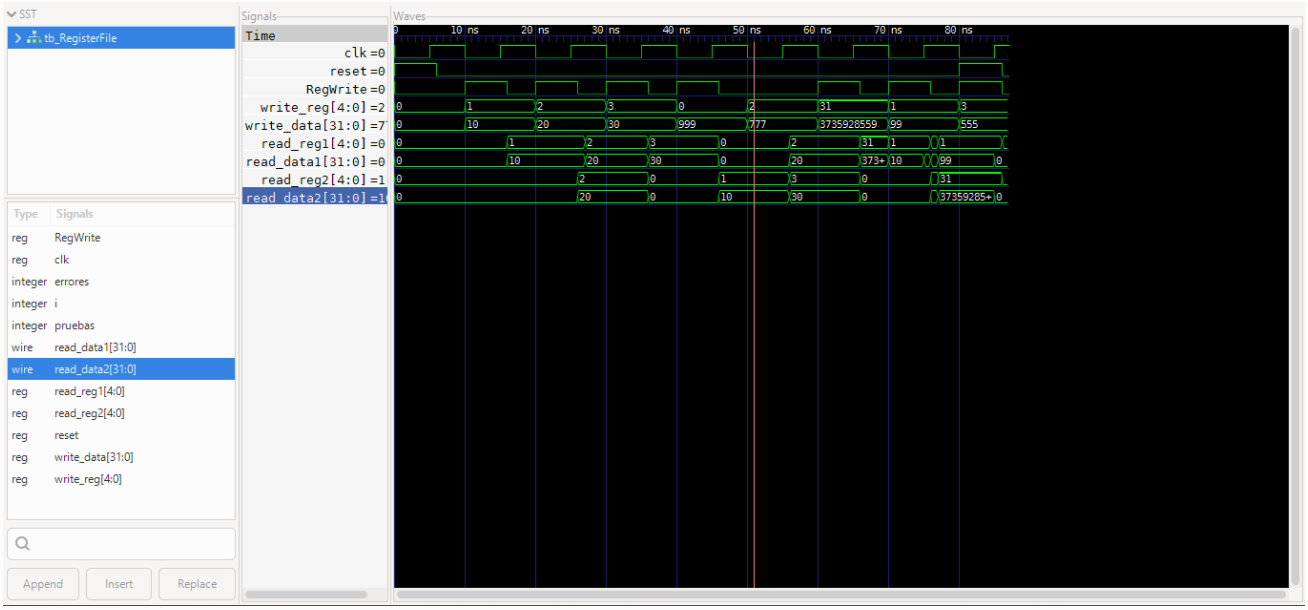
Captura — consola (Banco de Registros)

```
PS C:\Users\Usuario\Control_Registros> iverilog -s tb_RegisterFile -o RegisterFile_sim RegisterFile.v tb_RegisterFil
e.v
PS C:\Users\Usuario\Control_Registros> vvp RegisterFile_sim | Tee-Object -FilePath RegisterFile_consola.txt
VCD info: dumpfile RegisterFile.vcd opened for output.
=== Pruebas de RegisterFile ===
Reset inicial: 32 registros comprobados.
Escritura R1 <- 10 | R1=10 R2=0 R3=0 R0=0
Lectura: R1=10 | R0=0
Escritura R2 <- 20 | R1=10 R2=20 R3=0 R0=0
Lectura: R1=10 | R2=20
Lectura: R2=20 | R2=20
Escritura R3 <- 30 | R1=10 R2=20 R3=30 R0=0
Lectura: R3=30 | R0=0
Escritura R0 <- 999 | R1=10 R2=20 R3=30 R0=0
Lectura: R0=0 | R1=10
Lectura: R2=20 | R3=30
Escritura R31 <- 3735928559 | R1=10 R2=20 R3=30 R0=0
Lectura: R31=3735928559 | R0=0
Antes del flanco: R1=10, dato pendiente=99
Escritura sincronica: R1=99
Lectura: R2=20 | R3=30
Lectura: R1=99 | R31=3735928559
Lectura: R3=0 | R0=0
=== Fin: pruebas=87, errores=0 ===
tb_RegisterFile.v:209: $finish called at 87 (1ns)
```

4. Captura de GTKWave

Reloj, reset, RegWrite, direcciones, dato de escritura y las dos salidas de lectura. Que se note la escritura en el flanco ↑ y que la lectura cambia sin esperar el reloj.

Captura — GTKWave (Banco de Registros)



Señales del Banco de Registros. La escritura de R1 se refleja después del flanco positivo del reloj. Los cambios posteriores en las direcciones de lectura modifican las salidas sin esperar otro flanco. El reset final borra los registros en el siguiente flanco positivo.

5. Detalle de los casos de prueba

Describe qué haces en cada caso (dirección, dato y RegWrite), el resultado esperado a mano antes de simular y lo que dice la consola.

#	Caso de prueba	Qué haces (dirección · dato · RegWrite)	Esperado	Obtenido (consola)	¿OK?
1	**Reset** *Con reset = 1 y luego reset = 0, todos los registros valen 0*	Activar reset = 1 durante un flanco positivo, con RegWrite = 0; después desactivar reset.	Los 32 registros quedan en 0.	"Reset inicial: 32 registros comprobados", sin errores.	Sí
2	**Escribir y leer un registro** *Un registro distinto de $0*	Escribir 10 en R1 con RegWrite = 1 y leer R1.	R1 = 10.	R1 = 10.	Sí
3	**Dos registros a la vez** *Léelos por los dos puertos al mismo tiempo (y prueba el mismo registro en ambos)*	Tras escribir R1 = 10 y R2 = 20, leer ambos simultáneamente. Después seleccionar R2 en ambos puertos, con RegWrite = 0.	R1 = 10 y R2 = 20; después R2 = 20 en ambos puertos.	R1 = 10 y R2 = 20; después R2 = 20 en ambos puertos.	Sí
4	**Intento de escribir en $0** *Con RegWrite = 1; $0 debe seguir en 0*	Intentar escribir 999 en R0 con RegWrite = 1.	R0 permanece en 0.	R0 = 0.	Sí
5	**RegWrite = 0** *Cambian la dirección y el dato, pero no debe escribir nada*	Seleccionar R2 y write_data = 777 con RegWrite = 0 durante un flanco positivo.	R2 conserva 20.	R2 = 20.	Sí

#	Caso de prueba	Qué haces (dirección · dato · RegWrite)	Esperado	Obtenido (consola)	¿OK?
6	**Lectura asíncrona** *Cambia read_reg sin flanco de reloj: la salida cambia de inmediato*	Con RegWrite = 0, cambiar las direcciones de lectura de R2/R3 a R1/R31, sin esperar otro flanco.	Las salidas cambian de 20/30 a 99/3735928559.	R2 = 20, R3 = 30; después R1 = 99, R31 = 3735928559.	Sí
7	**Escritura síncrona** *El dato cambia entre flancos: se guarda lo que haya en el flanco ↑*	Con R1 = 10, preparar write_reg = 1, write_data = 99 y RegWrite = 1 antes del flanco positivo.	R1 conserva 10 antes del flanco y cambia a 99 después.	La comparación del valor previo pasó sin errores; después se imprimió "Escritura sincronica: R1=99".	Sí
8	**Sobrescribir** *Escribe dos veces el mismo registro*	Escribir primero 10 en R1 y posteriormente 99, con RegWrite = 1 en cada escritura.	La segunda escritura reemplaza 10 por 99.	Primero R1 = 10; después R1 = 99.	Sí
9	**Último registro** *Escribe y lee el $31*	Escribir 32'hDEADBEEF en R31 con RegWrite = 1 y leerlo.	R31 = 3735928559, equivalente a DEADBEEF hexadecimal.	R31 = 3735928559.	Sí
10	**Leer lo que se está escribiendo** *Mismo registro en lectura y escritura: ¿qué se ve antes y después del flanco?*	Leer R1 mientras se prepara una escritura de 99 en ese mismo registro.	Se lee 10 antes del flanco y 99 después.	Comparación previa sin errores y lectura posterior R1 = 99.	Sí
11	**Reset con datos** *Con registros escritos, activa reset: ¿qué pasa y cuándo?*	Con registros que contienen datos, activar reset = 1 y, simultáneamente, intentar escribir 555 en R3 con RegWrite = 1.	En el flanco positivo todos los registros quedan en 0; el reset impide guardar 555.	Comprobación de los 32 registros sin errores; lectura final R3 = 0 y R0 = 0.	Sí

Punto de control final — marca cada casilla

- ✓ Módulos y testbenches comentados, con los puertos del enunciado, $dumpfile y $dumpvars.
- ✓ Capturas de consola y de GTKWave de cada testbench, con los comandos y las señales legibles.
- ✓ Tablas de casos completas: esperado calculado a mano antes de simular, obtenido copiado de la consola.
- ✓ Decisiones documentadas (combinaciones no definidas, reset síncrono o asíncrono, cómo se protege $0).